Airfare Intelligence System

Project Report | Historical Fare Analytics and ML-Based Fare Estimation

1. Executive Summary

Airfare Intelligence System is a Python-based analytics application that turns a historical flight-fare dataset into an interactive dashboard. It combines exploratory data analysis, route and airline comparisons, configurable historical-flight filters, downloadable result tables, and a fare-estimation interface connected to a saved model artifact. The Streamlit application is running locally. Model prediction compatibility and public deployment should be treated as validation tasks until each has been tested end-to-end.

300,153	6	30	₹20,890
records shown by app	airlines	unique routes	mean recorded fare

2. Dataset and Scope

The application reads the local file airfare_data.csv. The currently displayed dataset summary reports 300,153 records, six airlines, 30 unique source–destination routes and a mean price of ₹20,890. These are application-reported figures, not independently recomputed statistics in this report. The dashboard code expects the following fields:

Field	Business meaning
airline / flight	Carrier and flight identifier
source_city / destination_city	Origin and destination city
departure_time / arrival_time	Time-of-day categories
stops	Number/category of stops
class	Cabin or travel class
duration	Flight duration, represented numerically
days_left	Days remaining before departure
price	Historical ticket fare used for analysis

3. Project Objectives

- Summarise historical fares and compare patterns by airline, route, class and booking window.
- Offer an accessible interface for filtering and reviewing matching historical flight records.
- Use a persisted machine-learning model to estimate a fare from selected flight attributes.
- Make the analysis reproducible through notebooks, a CSV dataset, a saved model file and a deployable Streamlit app.

4. Analytical and ML Workflow

The project folder contains three notebooks: 01_data_understanding.ipynb, 02_eda.ipynb and 03_preprocessing_modeling.ipynb. Their names indicate the workflow stages: inspect the dataset, explore patterns, then preprocess features and train/evaluate regression models. The selected model is persisted as best_model.joblib and loaded by the app. Specific model scores, train/test split details and the winning algorithm are not stated here because they have not been independently verified from the notebook outputs.

5. Application Components

Component	Implemented purpose
Dashboard	High-level KPIs; fare comparisons by airline; class distribution; booking-window trend; high-average-fare routes.
Fare Explorer	Filter by origin, destination, airline, class, departure time, stops, days left and fare range; view summary metrics and comparisons; sort and export filtered results to CSV.
Fare Estimator	Collect flight attributes and call the saved model to return an estimated fare, provided input features match the model pipeline.
Route Intelligence	Summarise route/airline records with average, median, minimum and maximum fares.
Dataset Explorer	Search the records, inspect a sample of rows, check missing-cell counts and export filtered data.

6. Technology Stack and Artefacts

Technology / file	Role
Python	Application logic and data processing
Pandas / NumPy	Dataframe operations and numerical calculations
Scikit-learn / Joblib	ML pipeline ecosystem and model persistence/loading
Streamlit	Interactive web application and navigation
Plotly	Interactive charts
airfare_data.csv	Historical dataset consumed by the app
best_model.joblib	Serialized trained-model artefact
Three Jupyter notebooks	Data understanding, EDA, preprocessing and modelling stages

7. Validation Status and Limitations

Confirmed from the latest user-provided screenshot: the Streamlit page opens locally, the sidebar navigation is visible, and the application displays the dataset summary metrics. A prior screenshot showed the chart-card headings but no visible plot content, so chart rendering should be checked after the page finishes loading.

The model file exists in the project folder and the current app code includes model loading and a prediction call; however, successful end-to-end prediction has not yet been demonstrated. The estimator must be tested against the exact training feature names, ordering, preprocessing and scikit-learn version. The dashboard works on historical data and does not provide live airline fares, inventory or booking.

8. Conclusion

Airfare Intelligence System packages historical flight-fare exploration, comparative analytics and a machine-learning estimation interface into a single Streamlit application. Its current local dashboard is operational. The next milestone is not additional styling; it is validating the model prediction path, checking charts and filters, and completing a clean deployment test. Performance claims should be added only after recording actual evaluation results from the modelling notebook.